Recursion!

If you think that the `flatten` function shown won't completely flatten deeply nested arrays such as `[1,[3,[4,5]]]`, then you're correct. Using recursion will be explored in chapter 6.

Before you had three similar functions to add, multiply, and flatten an array. Now you have one `accumulate` function that's an *abstraction* of the general idea of accumulating, or folding a function into an array or other data structure. The next important concept to learn for functions is *scope*. First, though, a quick mention of some syntax called *default arguments* that will tidy up some of what you've seen in this section.

Strategy pattern

If you have a background in design patterns from object-oriented programming languages, you might recognize that the way a function is passed in here is essentially a lightweight strategy pattern. If you don't know design patterns, you've just learned your first one!

Default argument values

The `accumulate` function uses a default value of `0` for the total. Instead of having to do this in the body of the function, you can use a default argument value. So, instead of

```
accumulate = (initial, numbers, accumulator) ->
  total = initial or 0
  # <rest of function omitted>
```

you can use

```
accumulate = (initial=0, numbers, accumulator) ->
  total = initial
  # <rest of function omitted>
```

Try a basic version of this on the REPL: